

Dataset Analysis Approach

March 21, 2025

 **sicor_municipal_co... .csv**
16.7 MB

Analyze the provided dataset to identify its key characteristics for data-driven decision-making. Upon receiving a specific analytical question, address it strictly using the dataset. Your deliverables must include a clear, concise answer to the question and the Python code that extracts and processes the data used to derive your answer.



qwen-max-latest

To provide a meaningful analysis and answer, I need a specific analytical question to address using the dataset. Please provide a question or a task that you'd like me to perform using the `sicor_municipal_contracts.csv` file.

For example, your question could be something like:

- "What is the total amount of contracts awarded for each municipal department?"
- "Which contractor received the highest total value of contracts?"
- "What is the average contract amount for each type of service?"

Once you provide a specific question, I'll analyze the data and write Python code to extract and process the relevant information to answer it.

How effective is the subprogram code as a feature for predicting industrialization-related operations? Define three approaches to evaluate its relevance and select the best one.



qwen-max-latest

To evaluate how effective the subprogram code is as a feature for predicting industrialization-related operations within the `sicor_municipal_contracts.csv` dataset, we can consider three distinct approaches. Each approach will provide insights into the relationship between the subprogram code and the target variable, which in this case pertains to industrialization-related operations.

Approach 1: Correlation Analysis

Description :

Compute the correlation between the subprogram code (assuming it's numerical or ordinal) and the contract amounts or other relevant numerical features indicative of industrialization activity. This could reveal whether specific subprogram codes are consistently associated with higher/lower values.

Steps :

1. Load the dataset.
2. Identify columns related to subprogram codes and industrialization indicators (e.g., contract amount).
3. Calculate the Pearson or Spearman correlation coefficient between the subprogram code and these indicators.

Pros :

- Quick and straightforward to implement.
- Provides a quantitative measure of association.

Cons :

- Only captures linear relationships.
- May not capture complex patterns if the relationship is non-linear.

Approach 2: Group-wise Statistical Analysis

Description :

Divide the data into groups based on subprogram codes and analyze the distribution of industrialization-related variables within each group. This can help identify patterns and differences across different subprograms.



Group the data by subprogram code and perform statistical tests (e.g., ANOVA, t-tests) to determine if there are significant differences in mean values of industrialization-related metrics across different subprogram codes.

Steps :

1. Load the dataset.
2. Group the data by subprogram code.
3. Perform an ANOVA test to compare means of a relevant metric (e.g., contract value) across groups.
4. Alternatively, conduct pairwise t-tests to identify specific pairs of subprogram codes that differ significantly.

Pros :

- Captures group-wise differences effectively.
- Can handle multiple categories and more complex comparisons.

Cons :

- Requires careful interpretation of p-values.
- Assumes normality and equal variances, which may not hold in all cases.

Approach 3: Machine Learning Feature Importance

Description :

Use a machine learning model to predict industrialization-related outcomes and assess the importance of the subprogram code as a feature. Tree-based models like Random Forests or Gradient Boosting Machines (e.g., XGBoost) are particularly useful since they provide feature importance scores.

Steps :

1. Load the dataset.
2. Preprocess the data to handle categorical variables, normalize/standardize numerical features, and split into training/test sets.
3. Train a predictive model (e.g., Random Forest) using subprogram code and other relevant features to predict an industrialization-related target.
4. Extract and analyze feature importance scores to gauge the relevance of the subprogram code.

Pros :

- Captures complex, non-linear relationships.
- Provides a holistic view of feature importance relative to other predictors.

Cons :

- Computationally intensive.
- Requires careful tuning and validation to avoid overfitting.

Selecting the Best Approach

Among these three approaches, **Approach 3: Machine Learning Feature Importance** is likely the best choice for evaluating the relevance of subprogram codes for predicting industrialization-related operations. This method offers several advantages:

1. **Comprehensive Insights** : It considers interactions among multiple features and captures complex patterns beyond simple correlations or group-wise differences.
2. **Feature Importance Scores** : Directly quantifies the contribution of the subprogram code relative to other features, providing clear guidance on its predictive power.
3. **Flexibility** : Can be adapted to various types of predictive models and evaluation metrics, accommodating different aspects of industrialization-related activities.

However, it is essential to ensure robust preprocessing and validation steps to derive reliable results. Here's a sample Python snippet demonstrating how you might implement Approach 3:

```
python
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.ensemble import RandomForestRegressor
4 from sklearn.preprocessing import StandardScaler
5
6 # Load dataset
7 file_path = 'sikor_municipal_contracts.csv'
8 data = pd.read_csv(file_path)
```



```

9
10 # Assuming 'subprogram_code' is a column and 'contract_amount' is the target
11 X = data[['subprogram_code', 'other_relevant_feature_1', 'other_relevant_feature_2']]
12 y = data['contract_amount']
13
14 # Preprocess data (handle missing values, encode categoricals if necessary)
15 scaler = StandardScaler()
16 X_scaled = scaler.fit_transform(X)
17
18 # Split data
19 X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
20
21 # Train Random Forest model
22 model = RandomForestRegressor(random_state=42)
23 model.fit(X_train, y_train)
24
25 # Extract feature importances
26 feature_importances = model.feature_importances_
27 features = X.columns
28 importance_df = pd.DataFrame({'Feature': features, 'Importance': feature_importances}).sort_valu
29
30 print(importance_df)

```

This script outlines the basic steps: loading the data, preprocessing, splitting into training and testing sets, training a Random Forest model, and extracting feature importances. Adjust the preprocessing steps and feature selection based on your specific dataset structure and requirements.

